Date:15/10/2026

# CO1-AT3-APPLICATIONS BASED QUESTIONS

# COURSE CODE: DSA0308

# COURSE NAME:NATURAL LANGUAGE AND PREPROCESSING

REG NO:192424322 NAME:M.ARSHINI

ANSWERS:

## Problem 1: Intelligent Email and Password Validator Using Regular Expressions

### 1. Problem Identification

The objective is to develop a Python program using Regular Expressions (Regex) to validate three types of user information:

- Email address
- Password strength
- Mobile number

The program checks the input against the rules specified in your assignment.

### 2. Functional requirements

| Input | Validation rules |
|---|---|
| Email | Starts with a letter; before @, allows letters, digits, dots and underscores; domain contains only letters; extension must be .com, .org, .edu, .net or .in |
| Password | At least 8 characters, one uppercase letter, one lowercase letter, one digit and one special character from @ # $ % & ! |
| Mobile number | Exactly 10 digits, starting with a digit from 6 to 9 |

### 3. Algorithm

1. Start the program.
2. Import Python's re module.
3. Read the email, password and mobile number from the user.
4. Use a regular expression to validate the email address.
5. Check the password for the required length and character types.

6. Validate the mobile number using a regular expression.
7. Display the result for each input.
8. Stop the program.

**4. Python program**

```
import re


def validate_email(email):
    pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
    return re.fullmatch(pattern, email, re.IGNORECASE) is not None


def validate_password(password):
    pattern = (
        r"^(?=.*[A-Z])"
        r"(?=.*[a-z])"
        r"(?=.*\d)"
        r"(?=.*[@#$%&!])"
        r".{8,}$"
    )
    return re.fullmatch(pattern, password) is not None


def validate_mobile(mobile):
    pattern = r"^[6-9]\d{9}$"
    return re.fullmatch(pattern, mobile) is not None


def main():
```

```
    print("=== Email, Password and Mobile Validator ===")

    email = input("Enter email address: ").strip()
    password = input("Enter password: ")
    mobile = input("Enter mobile number: ").strip()

    print("\n--- Validation Results ---")

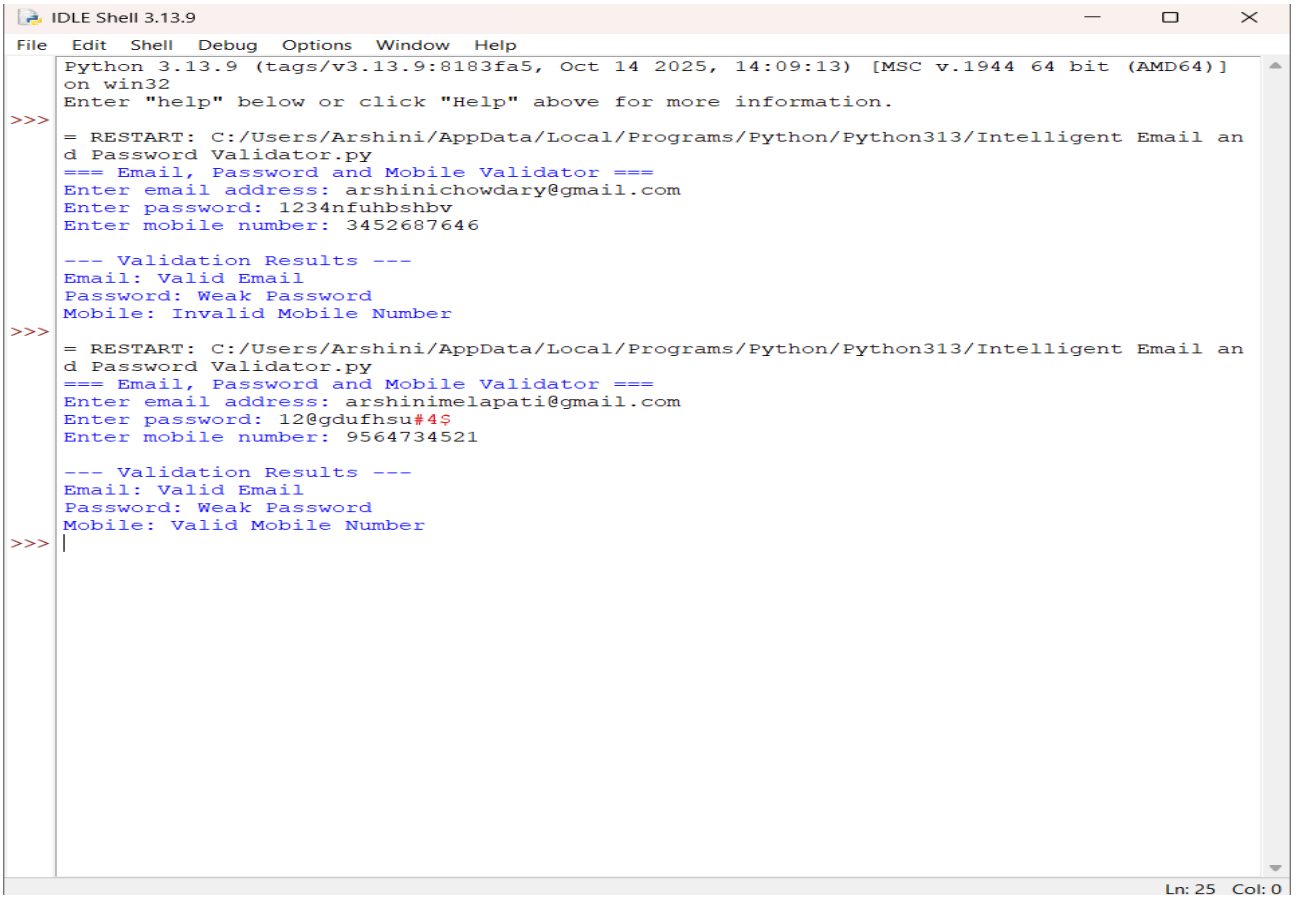
    if validate_email(email):
        print("Email:", "Valid Email")
    else:
        print("Email:", "Invalid Email")

    if validate_password(password):
        print("Password:", "Strong Password")
    else:
        print("Password:", "Weak Password")

    if validate_mobile(mobile):
        print("Mobile:", "Valid Mobile Number")
    else:
        print("Mobile:", "Invalid Mobile Number")


if __name__ == "__main__":
    main()
```

# Output:


IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Arshini/AppData/Local/Programs/Python/Python313/Intelligent Email and Password Validator.py
=== Email, Password and Mobile Validator ===
Enter email address: arshinichowdary@gmail.com
Enter password: 1234nfuhbshbv
Enter mobile number: 3452687646
--- Validation Results ---
Email: Valid Email
Password: Weak Password
Mobile: Invalid Mobile Number
>>>
= RESTART: C:/Users/Arshini/AppData/Local/Programs/Python/Python313/Intelligent Email and Password Validator.py
=== Email, Password and Mobile Validator ===
Enter email address: arshinimelapati@gmail.com
Enter password: 12@gdufhsu#4$
Enter mobile number: 9564734521
--- Validation Results ---
Email: Valid Email
Password: Weak Password
Mobile: Valid Mobile Number
>>>
Ln: 25 Col: 0

## Problem 2: Design a Finite State Automata (FSA) Simulator

### 1. Problem Identification

A Deterministic Finite Automaton (DFA) is a mathematical model used to recognize patterns in strings. For each current state and input symbol, the DFA has exactly one next state.

Problem statement: Develop a Python program that simulates a DFA and checks whether input strings belong to a specified language.

The example language in your assignment is strings ending with "ab".

### 2. Requirements

The program should accept:

1. Set of states.
2. Input alphabet.
3. Transition table.
4. Initial state.
5. Final state or states.
6. Multiple input strings.

It should simulate the transitions for each string, display the transition path, and print Accepted or Rejected.

### 3. Algorithm

1. Start the program.
2. Read the DFA states and input alphabet.
3. Read the transition table for each state and symbol.
4. Read the initial state and final states.
5. Accept multiple input strings.
6. Set the current state to the initial state.
7. Read each character in the input string and follow the corresponding transition.
8. Record each visited state in the transition path.
9. If any symbol has no defined transition, reject the string.
10. After processing all characters, check whether the current state is a final state.
11. Display the transition path and result.

12. Repeat for all input strings and stop

## 4. Python program

```
def simulate_dfa(string):
    # DFA for strings ending with "ab"
    transitions = {
        "q0": {"a": "q1", "b": "q0"},
        "q1": {"a": "q1", "b": "q2"},
        "q2": {"a": "q1", "b": "q0"}
    }

    initial_state = "q0"
    final_states = {"q2"}

    current_state = initial_state
    path = [current_state]

    for symbol in string:
        if symbol not in {"a", "b"}:
            return path, False, "Invalid input symbol"

        current_state = transitions[current_state][symbol]
        path.append(current_state)

    accepted = current_state in final_states
    return path, accepted, None



def main():
    print("=== DFA Simulator ===")
    print("Language: Strings ending with 'ab'")
```

```
    print("Alphabet: a, b")

    try:
        count = int(input("How many strings to test? "))

        if count < 1:
            print("Enter a positive number.")
            return

        for i in range(count):
            string = input(f"\nEnter string {i + 1}: ")

            path, accepted, error = simulate_dfa(string)

            print("Transition Path:", " -> ".join(path))

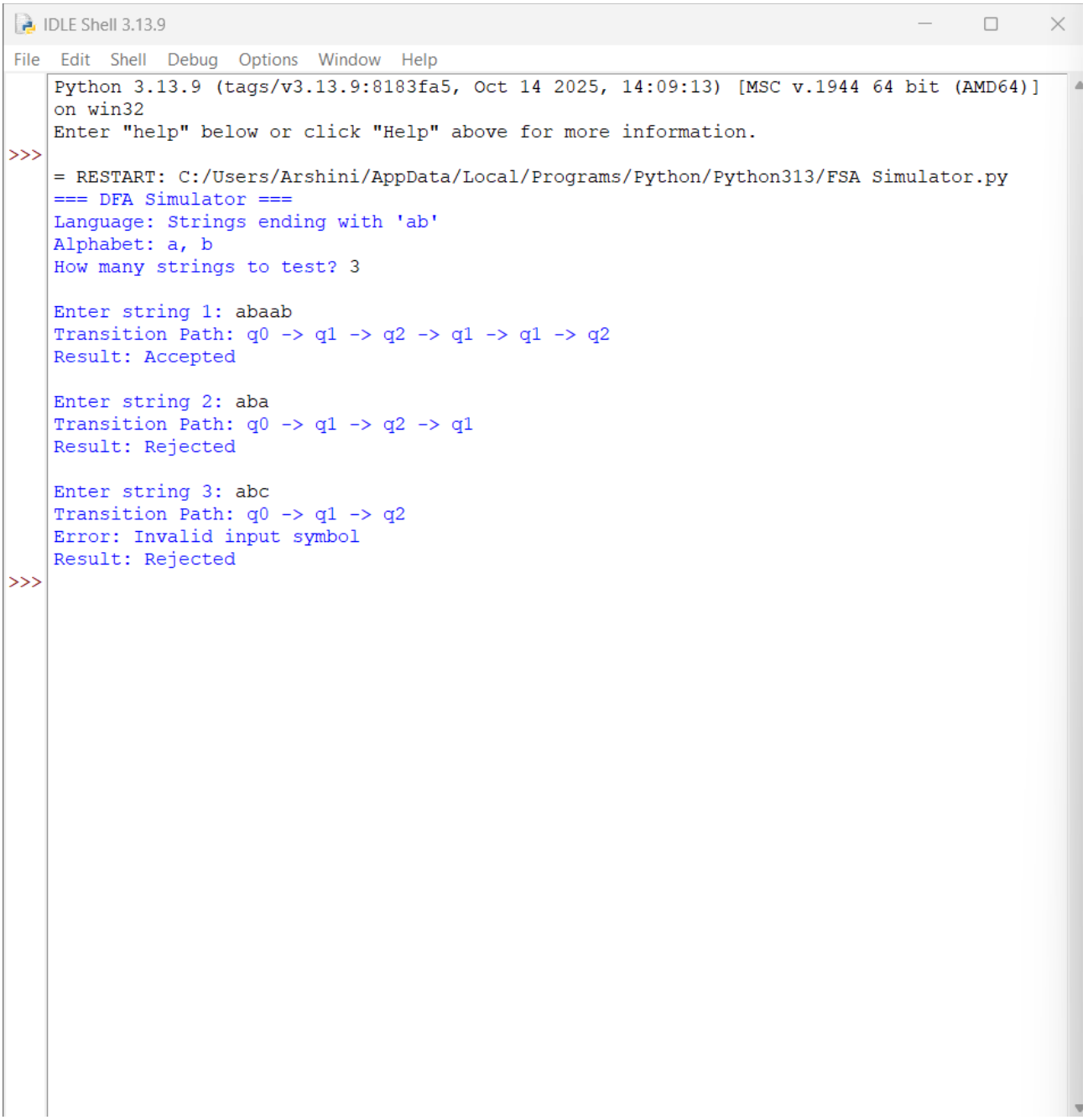
            if error:
                print("Error:", error)

            print("Result:", "Accepted" if accepted else "Rejected")

    except ValueError:
        print("Invalid input. Enter a valid number of strings.")


if __name__ == "__main__":
    main()
```

## Output:


IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Arshini/AppData/Local/Programs/Python/Python313/FSA Simulator.py
=== DFA Simulator ===
Language: Strings ending with 'ab'
Alphabet: a, b
How many strings to test? 3
Enter string 1: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Result: Accepted
Enter string 2: aba
Transition Path: q0 -> q1 -> q2 -> q1
Result: Rejected
Enter string 3: abc
Transition Path: q0 -> q1 -> q2
Error: Invalid input symbol
Result: Rejected
>>>

## Problem 3: Smart Pattern Matching Engine Using Regular Expressions

### 1. Problem Identification

Develop a Python-based text search engine that performs pattern matching using Regular Expressions (Regex).

The system should support the following operations, as specified in your assignment:

1. Word Search
2. Prefix Search
3. Suffix Search
4. Date Search
5. Phone Number Search
6. Hashtag Search
7. Mention (@username) Search

### 2. Algorithm

1. Start the program.
2. Read the text from the user.
3. Display the search menu.
4. Ask the user to select a search operation.
5. If the user selects date search, identify dates using a regular expression.
6. If the user selects phone number search, identify phone numbers using a regular expression.
7. If the user selects hashtag search, identify hashtags beginning with #.
8. If the user selects mention search, identify usernames beginning with @.
9. If the user selects prefix search, find words starting with the specified prefix.
10. If the user selects suffix search, find words ending with the specified suffix.
11. If the user selects word search, find occurrences of the specified word.
12. Display all matching results or a message if no matches are found.
13. Repeat the process until the user chooses to exit.
14. Stop the program.

## Python code:

```
import re
def search_word(text, word):
    pattern = r"\b" + re.escape(word) + r"\b"
    return re.findall(pattern, text, re.IGNORECASE)
def search_prefix(text, prefix):
    words = re.findall(r"\b\w+\b", text)
    return [word for word in words
            if word.lower().startswith(prefix.lower())]
def search_suffix(text, suffix):
    words = re.findall(r"\b\w+\b", text)
    return [word for word in words
            if word.lower().endswith(suffix.lower())]
def search_date(text):
    pattern = r"\b\d{2}/\d{2}/\d{4}\b"
    return re.findall(pattern, text
def search_phone(text):
    pattern = r"\b[6-9]\d{9}\b"
    return re.findall(pattern, text)
def search_hashtag(text):
    pattern = r"(?<!\w)#[A-Za-z0-9_]+"
    return re.findall(pattern, text)
def search_mention(text):
    pattern = r"(?<!\w)@[A-Za-z0-9_]+"
    return re.findall(pattern, text)
def main():
    print("=== Smart Pattern Matching Engine ===")

    text = input("Enter text: "
    while True:
```

```
print("\n--- Search Menu ---")
print("1. Search Word")
print("2. Search Prefix")
print("3. Search Suffix")
print("4. Search Date")
print("5. Search Phone Number")
print("6. Search Hashtag")
print("7. Search Mention")
print("8. Exit")

choice = input("Enter your choice: ").strip()

if choice == "1":
    word = input("Enter word: ")
    result = search_word(text, word)

elif choice == "2":
    prefix = input("Enter prefix: ")
    result = search_prefix(text, prefix)

elif choice == "3":
    suffix = input("Enter suffix: ")
    result = search_suffix(text, suffix)

elif choice == "4":
    result = search_date(text)

elif choice == "5":
    result = search_phone(text)
```

```
        elif choice == "6":
            result = search_hashtag(text)
        elif choice == "7":
            result = search_mention(text)
        elif choice == "8":
            print("Exiting program.")
            break
        else:
            print("Invalid choice. Please enter 1 to 8.")
            continue
        if result:
            print("Matching patterns:", ", ".join(result))
        else:
            print("No matching patterns found.")
if __name__ == "__main__":
    main()
```

## Output:


*IDLE Shell 3.13.9*
File Edit Shell Debug Options Window Help
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Arshini/AppData/Local/Programs/Python/Python313/Smart Pattern Matching.py
=== Smart Pattern Matching Engine ===
Enter text: Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural language processing
--- Search Menu ---
1. Search Word
2. Search Prefix
3. Search Suffix
4. Search Date
5. Search Phone Number
6. Search Hashtag
7. Search Mention
8. Exit
Enter your choice: 4
Matching patterns: 12/09/2026
--- Search Menu ---
1. Search Word
2. Search Prefix
3. Search Suffix
4. Search Date
5. Search Phone Number
6. Search Hashtag
7. Search Mention
8. Exit
Enter your choice: 6
Matching patterns: #NLP
--- Search Menu ---
1. Search Word
2. Search Prefix
3. Search Suffix
4. Search Date
5. Search Phone Number
6. Search Hashtag
7. Search Mention
8. Exit
Enter your choice:
Ln: 41 Col: 19